

تمرینات اضافی

درس برنامه‌نویسی پیشرفته

دانشکده مهندسی برق، دانشگاه شهید بهشتی

زمستان 1403

1) دو رشته s_1 و s_2 از کاربر دریافت شوند. کوچکترین بازه‌ای در رشته s_1 را بیابید که شامل تمام کاراکترهای رشته s_2 (شامل تکرارها) باشد. در صورتی که چنین بازه‌ای وجود نداشته باشد، یک رشته خالی ("") بازگردانید. اگر چندین بازه با طول یکسان وجود داشت، بازه‌ای را که زودتر شروع می‌شود باز گردانید. (تمام کاراکترها با حروف کوچک lowercase هستند)
مثال:

ورودی: $s_1 = \text{"timetopractice"}$ و $s_2 = \text{"toc"}$
خروجی: "toprac"

ورودی: $s_1 = \text{"zoomlzapzo"}$ و $s_2 = \text{"oza"}$
خروجی: "apzo"

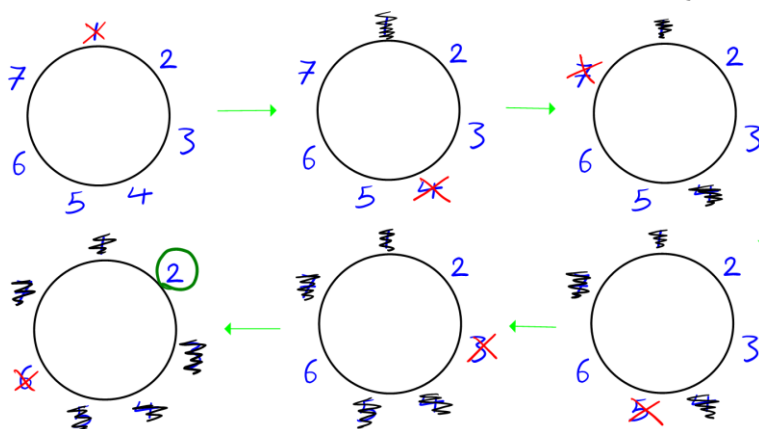
ورودی: $s_1 = \text{"zoom"}$ و $s_2 = \text{"zooe"}$
خروجی: ""

بارم: 0.33

(2) فرض کنید n نفر دور یک میز گرد نشسته‌اند و به صورت ساعتگرد افراد را از 1 تا n شماره‌گذاری کرده‌ایم. یک عدد طبیعی کوچکتر از n مانند k در نظر بگیرید. سپس یک الگوریتم اجرا می‌شود؛ ابتدا نفر اول میز را ترک می‌کند، k نفر بعد پشت میز می‌مانند و بعد نفر $k+1$ میز را ترک می‌کند دوباره k نفر از آن باقی می‌مانند و نفر بعدی میز را ترک می‌کند و الگوریتم به همین ترتیب ادامه پیدا می‌کند. سوال اینجاست که در نهایت آخرین نفری که پشت میز می‌ماند کیست؟ برنامه‌ای بنویسید که k و n را از کاربر دریافت کرده و در نهایت شماره فردی که در انتها باقی می‌ماند را نمایش دهد.

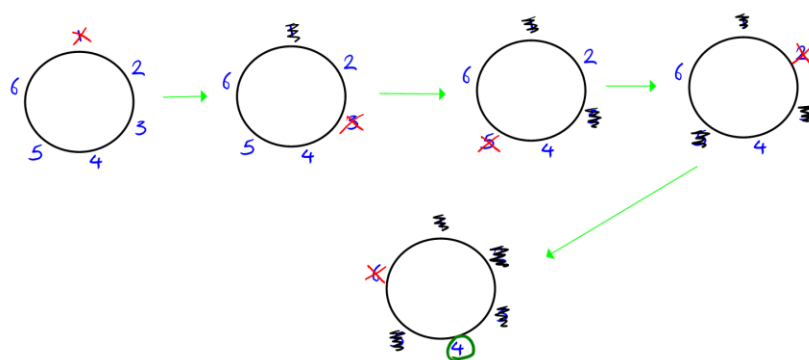
مثال:

ورودی: $n=7$ و $k=2$



خروجی: 2

ورودی: $n=5$ و $k=1$



خروجی: 4

بارم: 0.25

(3) دو رشته از کاربر دریافت شوند. یکی رشته متن `txt` و دیگری رشته الگو `pat`. هدف این است که اندیس کاراکتر آغازین جاهایی از متن `txt` که متن `pat` در آن جاها ظاهر شده است را نمایش دهد. توجه داشته باشید که برنامه به حروف بزرگ و کوچک متن‌ها حساس نباشد. (یعنی بزرگ و کوچک معادل در نظر گرفته شود)

مثال:

ورودی: `pat = "aB"` و `txt = "aBcAb"`
خروجی: `[0, 3]`

ورودی: `pat = "Aa"` و `txt = "aAAa"`
خروجی: `[0, 1, 2]`

ورودی: `pat = "aC"` و `txt = "abba"`
خروجی: `[]`

بازم: 0.25

4) یک عدد صحیح n از کاربر دریافت کنید. برنامه‌ای بنویسید که یک آرایه ans با اندازه $n+1$ خروجی دهد که عنصر i ام آن نشان‌دهنده تعداد 1های موجود در نمایش باینری عدد i باشد. ($0 \leq i \leq n$)
مثال:

ورودی: $n=2$

خروجی: $[0, 1, 1]$

ورودی: $n=5$

خروجی: $[0, 1, 1, 2, 1, 2]$

$0 \rightarrow 0$

$1 \rightarrow 1$

$2 \rightarrow 10$

$3 \rightarrow 11$

$4 \rightarrow 100$

$5 \rightarrow 101$

0.25 بارم:

(5) یک برنامه بنویسید که دو عدد 4 رقمی اول از کاربر دریافت کند، Num1 و Num2. برنامه باید فاصله کوتاه‌ترین مسیر از Num1

به Num2 را پیدا کند و آن را نمایش دهد. یک مسیر از Num1 به Num2 یعنی در هر مرحله تنها یک رقم از عدد قبلی را عوض کنیم (به بیان دیگر عدد بعدی از تغییر تنها یک رقم عدد قبلی به دست می‌آید) و همچنین در هر مرحله باید عدد به دست آمده اول باشد. (اگر احیاناً بین دو عدد با شرایط گفته شده مسیری وجود نداشت برنامه عبارت no path را خروجی دهد).
مثال:

ورودی: Num1 = 1033 و Num2 = 8179

خروجی: 6

مسیر: 1033 → 1733 → 3733 → 3739 → 3779 → 8779 → 8179

ورودی: Num1 = 1033 و Num2 = 1033

خروجی: 0

بارم: 0.33

6) یک برنامه طراحی کنید که دارای یک کلاس `DynamicArray` باشد که دارای یک آرایه پویا برای ذخیره اعداد است. همچنین این کلاس یک `Constructor` داشته باشد که اندازه آرایه را دریافت کرده و آرایه را به صورت پویا اختصاص دهد. همچنین لازم است یک `copy constructor` برای اطمینان از کپی عمیق (`deep copy`) پیاده‌سازی شود. این کلاس باید یک `Destructor` داشته باشد که حافظه اختصاص داده شده به آرایه را آزاد کند (پاک کند) `(delete)`.
مهمتر از همه، این کلاس باید یک تابع داشته باشد که با هر بار فراخوانی، اعداد آرایه را از کوچک به بزرگ مرتب کند.

بارم: 0.25

7) یک سیستم مدیریت کتابخانه طراحی کنید که بتواند اطلاعات مربوط به آیتم‌های مختلف کتابخانه (مانند کتاب‌ها و مجلات) را مدیریت کند. یک کلاس پایه به نام `Item` ایجاد کنید که شامل ویژگی‌های عمومی همه آیتم‌ها، مانند: عنوان، نویسنده، شماره شابک و ... باشد.

دو کلاس مشتق‌شده به نام‌های `Book` و `Magazine` تعریف کنید: 1- کلاس `Book` باید شامل اطلاعاتی درباره تعداد صفحات (`page count`) باشد. 2- کلاس `Magazine` باید شامل اطلاعاتی درباره شماره انتشار (`issue number`) باشد. در کلاس `Item` یک متد مجازی به نام `displayInfo()` تعریف کنید که اطلاعات آیتم را نمایش دهد. این متد در کلاس‌های `Book` و `Magazine` بازنویسی (`override`) شود تا اطلاعات مخصوص هر نوع آیتم را نمایش دهد. در برنامه اصلی، مجموعه‌ای از آیتم‌ها (ترکیبی از کتاب‌ها و مجلات) را در یک آرایه یا لیست از اشاره‌گرهای کلاس پایه `Item` ذخیره کنید و با فراخوانی متد `displayInfo()` اطلاعات هر آیتم را نمایش دهید.

بارم: 0.25

(8) برنامه‌ای بنویسید که برای انجام اعمال ریاضی روی اعداد خیلی بزرگ مناسب باشد. یک کلاس `HugeInteger` تعریف کنید که بتواند یک عدد (ممکن است عدد تا حدی بزرگ باشد که تعداد ارقام آن نهایتاً تا 50 باشد) را به صورت یک آرایه از ارقام ذخیره کند.

همچنین در این کلاس توابع مربوط به اعمال جمع دو عدد، تفریق دو عدد، چک کردن برابر بودن دو عدد و ضرب دو عدد وجود داشته باشد و بتوانیم این اعمال را روی اعداد بزرگ (تا 50 رقم) انجام دهیم.

بارم: 0.33

(9) یک ماتریس داده شده است که هر خانه آن می‌تواند مقادیر 0، 1 یا 2 داشته باشد که به شرح زیر است:

0: خانه خالی

1: خانه حاوی پرتقال تازه

2: خانه حاوی پرتقال فاسد

باید تعیین کنید که حداقل زمان لازم برای فاسد شدن تمام پرتقال‌های تازه چقدر است.

یک پرتقال فاسد در خانه $[i, j]$ می‌تواند پرتقال‌های تازه همسایه خود در خانه‌های مجاور یعنی بالا، پایین، چپ و راست را در یک واحد زمان فاسد کند.

برنامه‌ای بنویسید که وضعیت اولیه ماتریس را دریافت کند. به عنوان خروجی حداقل زمان لازم برای فاسد شدن همه پرتقال‌ها را گزارش کند. (اگر فاسد شدن همه پرتقال‌ها امکان‌پذیر نباشد برنامه عدد 1- را بازگرداند)
مثال:

$$\begin{bmatrix} 0 & 1 & 2 \\ 0 & 1 & 2 \\ 2 & 1 & 1 \end{bmatrix} \text{ ورودی:}$$

خروجی: 1

$$\begin{bmatrix} 2 & 2 & 0 & 1 \end{bmatrix} \text{ ورودی:}$$

خروجی: 1-

$$\begin{bmatrix} 2 & 2 & 2 \\ 0 & 2 & 0 \end{bmatrix} \text{ ورودی:}$$

خروجی: 0 (چون از ابتدا همه پرتقال‌ها فاسد هستند).

بارم: 0.25